Comparison of Impact of Data Dirtiness and Imputation Techniques on Model Performance

Ksenia Krasikova[1*†], **Payal Priyadarshini**[1*†], **Philipp Harlos**[1*†]

[1] Department of Mathematics and Informatics, FU Berlin, Berlin, DE

***For correspondence:**
ksek00@zedat.fu-berlin.de (KK);
payap02@zedat.fu-berlin.de (PP);
harlop97@zedat.fu-berlin.de (QQ)

[†]These authors contributed equally to this work

Abstract

Goal of the project: Comparing the robustness and importance of the features of two machine learning models in different datasets, each with different levels of data dirtiness, all derived from the same source data set.
Main results: Subsiding performancee of the models and detection of original features with higher degrees of distortion.
Personal key learnings: Implementation of data imputation methods and understanding of the impact of data degradation.
Estimated working hours: 7
Project evaluation: 2
Number of Words: 1444

Goal

As a continuation of last week's work, our objective was to evaluate the robustness of two models, Random Forest and linear kernel SVM, under controlled data degradation.
We will degrade the data under different conditions and then apply cleaning strategies to recover it. By doing this, we obtain multiple datasets under different conditions, on which we will compare each model's performance using the area under the ROC curve (AUC) and accuracy scores. Lastly, we compare the extracted top five metabolites by feature importance.

Data

We reused the MTBLS90 dataset from the last week project ***Ganna et al.*** (***2014***), which contains liquid chromatography mass spectrometry measurements of 189 metabolites in 968 plasma samples from 70-year-old participants (483 females, 485 males). To evaluate the robustness of the model under data degradation, we applied two 'dirty' strategies - missing values and added noise - each at two levels (10 % and 50 %). This setup lets us compare both different strategies and the impact of low versus high degradation within each strategy.

Missing Values

We introduced missing values to simulate incomplete datasets. Each cell in the dataset was represented by a unique index. Using the NumPy package (***Harris et al., 2020***), indices were randomly selected, and their corresponding values were replaced with `NaN`.

Adding Noise

For our second strategy, we introduced numerical noise to simulate measurement errors and to increase dataset variance. Similar to the missing values strategy, cells were randomly selected using NumPy indices. The noise was randomly generated from a uniform distribution ranging between 10% and 90% of the selected cell's original value and was randomly applied either as an increase or a decrease.

Data Preprocessing

For each of the two "missing-value" datasets (10% and 50% of values replaced by NaN), we applied three cleaning strategies:

1. **Mean imputation** via scikit-learn's (***Pedregosa et al., 2018***) SimpleImputer (strategy="mean"), which replaces each missing entry with its column's average.
2. **K-nearest neighbours imputation** using scikit-learn's KNNImputer (n_neighbors=5), which infers a missing value from its five closest samples.
3. **Row deletion**, dropping any sample that contains at least one NaN.

In principle, this should have produced six "cleaned" datasets, but the two variants involving row deletion resulted in empty datasets. This is an expected outcome because even adding 10% missing values across all cells produces around 18295 missing entries. Given 986 rows, this leads to a high probability that a row contains at least one missing value. We discarded the two empty data sets and then trained and evaluated on the remaining four, as well as on the two "added noise" datasets and the untouched original dataset. This produces seven data variants in total. This process is also shown in ***Figure A1***.

Data Analysis

After data preprocessing, we train two machine learning models, Random Forests and Support Vector Machine with linear kernel, on the seven remaining datasets. Model training and downstream evaluation were implemented using scikit-learn (***Pedregosa et al., 2018***).

Random Forests

Random Forests is a method that constructs multiple decision trees from the data and combines their results to predict the output (***Chen et al., 2013***). For the model implementation, RandomForestClassifier from scikit learn was used. Model parameters were obtained from previous hyperparameter optimization in Project 1. The remaining parameters are also set according to the last project. We refrained from performing hyperparameter optimization for the Random Forest models, as the increased number of datasets combined with a larger hyperparameter search space would make the training process more time-consuming.

For each dataset, a separate model, however, with the same initial settings, is trained and evaluated iteratively. After training, we identify the top five most important features for each model variant using the feature_importances_ attribute, which quantifies the contribution of each feature to the model's predictive performance based on the total decrease in node impurity (***Chen et al., 2013***).

Support Vector Machine - linear

Support Vector Machines (SVM) is an algorithm that finds a hyperplane to distinctly separate the data, where different kernels change how the data is transformed to ensure separation (***Mahadevan et al., 2008***). For model implementation, the SVC from scikit-learn was used. Here we use an SVM with a linear kernel. The regularization parameter c was selected through hyperparameter optimization using GridSearchCV. The remaining parameters are set according to the last project. Similar to the training of the Random Forests models, each SVM is trained and evaluated iteratively.

Lastly, we extract the absolute feature importance using coef_ attribute. These values reflect the absolute contribution of each feature to the decision boundary (***Mahadevan et al., 2008***).

Evaluation

After training, AUC_{train} and AUC_{test} are computed, and ROC curves are plotted using functions roc_auc_score and roc_curve from scikit-learn. Additionally, for each model and dataset, classification accuracy is evaluated using the accuracy_score function.

Results & Discussion

Model Metrics

First, for the Random Forests model, the following results are obtained after training and evaluation completion: the model for the original dataset achieved the best AUC_{test} and accuracy, followed by models for 10% imputed datasets. These have similar AUC_{test} but the KNN-imputed data resulted in slightly better accuracy for the model. As for 50% imputed datasets, the RF model performed better with the mean imputed data. Lastly, the performance of models on 10% wrong data is similar to the performance of the model for 50% mean imputed data, with slightly better accuracy. However, for 50% wrong data, the AUC_{test} and accuracy drop visibly compared to all other datasets, especially seen in the plot of the ROC curves ***Figure A2***.

Remarkably, with a progressive degree of degradation, AUC_{train} scores stay the same and even get higher, up to a score of 1. These high AUC_{train} values compared to lower AUC_{test} and accuracy scores suggest that the RF models tend to overfit on this data and overfit even more when more noise is added, either by imputed missing values or wrong data entries.

For SVM, this pattern of performance is similar, as the model trained on original data performs the best, followed by 10% and then 50% imputed data. However, different from the RF model, for both models for data sets that had noisy data, AUC_{test} and accuracy drop significantly compared to others. These values, slightly higher than 0.5, suggest that for noisy, uncorrected data, linear SVM models perform as well as randomly assigning the binary labels.

As for overfitting, for all SVM models, except the two trained on noisy data, AUC_{train}, AUC_{test} and accuracy scores show no signs of overfitting.

Important Features

Next, we examine the elucidated important features. In Random Forests models, the original and 10% mean and KNN imputed, the top 4 features were the same and in retained order, with only the fifth feature differing. The remaining four models retained three to four original features, but in a different order of importance.

The linear SVM models show a similar pattern regarding the feature importance rankings. In the model trained on 10% KNN imputed data, the top four features are retained in the same order. The 10% mean imputed and both 50% imputed data models retain three to four original features but in a shuffled order. However, for the remaining two models trained on noisy data, no original features are obtained. This follows the pattern of those two models being remarkably different in their performance compared to the RF models trained on the same data and other SVM models.

Main Insights

Overall, with a higher degree of "dirtiness", the performance of the two inspected models gets worse, measured in AUC_{test} and accuracy. Especially, this effect is seen on noisy data, where the values are not being corrected, as there is no way to detect the true values. However, RF models show more stability when it comes to noisy data, as linear SVM has no predictive value on these two datasets. Between the two imputation methods, KNN imputation works slightly better when there is less missing data, whereas mean imputation leads to slightly better performance on higher degrees of degradation in both inspected models.

Although the original predictive features are still elucidated in part of the models, the order and thus predictive importance vary, especially with degradation progression and in all noisy data SVM models, where none of the originally predictive features retained their value. These differences in prediction are accounted for by imputation and noise in the data: as imputation may restore missing values, it changes original values where some features may become more important than before. The same can be said for added noise: by chance, this may enhance one feature and reduce the importance of others. These facts have to be accounted for when those features are the goal of interpretation, as interpretation may change when different features become important and thus different metabolites are examined.

Contributions

1. Data and Random Forest model was implemented by Ksenia Krasikova.
2. The SVM model was implemented by Payal Priyadarshini across seven distinct datasets.
3. Mainly report Writing (Section: Goal, Data, Data Preprocessing, Figure/Table creation) by Philipp Harlos

References

Chen T, Cao Y, Zhang Y, Liu J, Bao Y, Wang C, Jia W, Zhao A. Random Forest in Clinical Metabolomics for Phenotypic Discrimination and Biomarker Selection. Evidence-Based Complementary and Alternative Medicine. 2013; 2013:1–11. http://dx.doi.org/10.1155/2013/298183, doi: 10.1155/2013/298183.

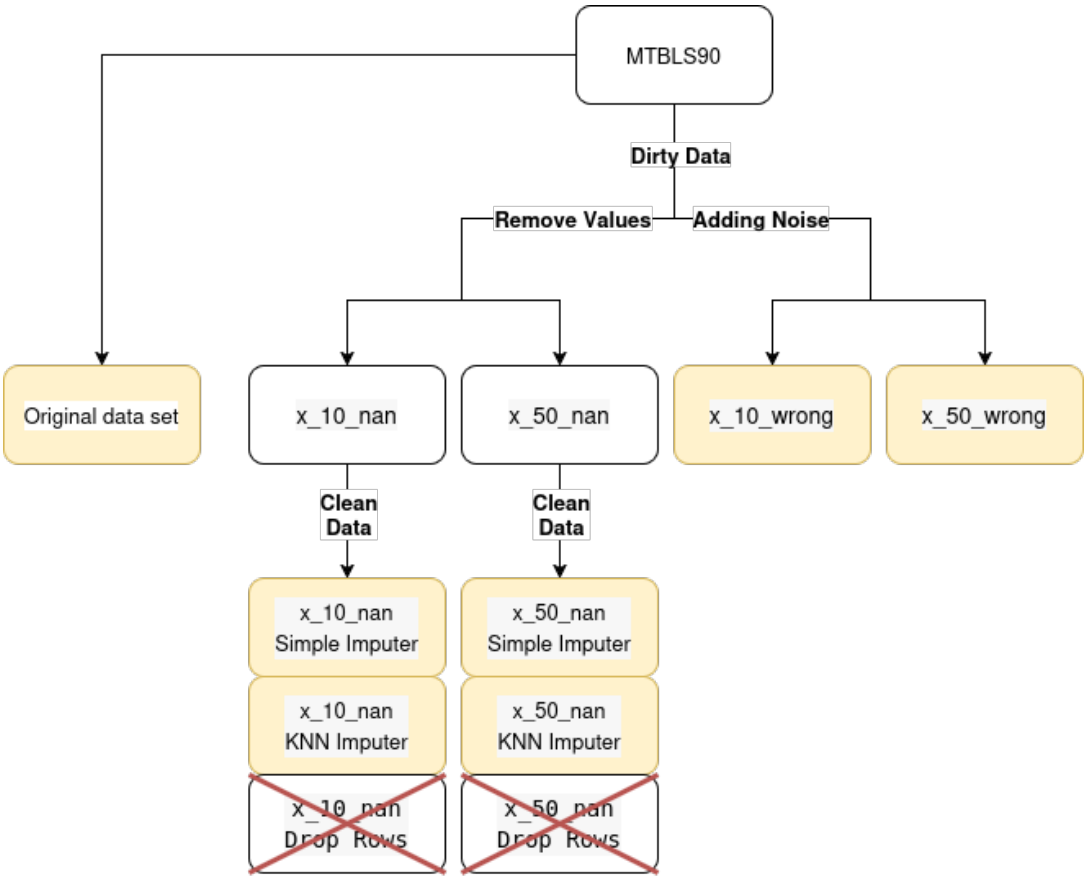
Ganna A, Salihovic S, Sundström J, Broeckling CD, Hedman , Magnusson PKE, Pedersen NL, Larsson A, Siegbahn A, Zilmer M, Prenni J, Ärnlöv J, Lind L, Fall T, Ingelsson E. Large-scale Metabolomic Profiling Identifies Novel Biomarkers for Incident Coronary Heart Disease. PLoS Genetics. 2014 Dec; 10(12):e1004801. http://dx.doi.org/10.1371/journal.pgen.1004801, doi: 10.1371/journal.pgen.1004801.

Harris CR, Millman KJ, van der Walt SJ, Gommers R, Virtanen P, Cournapeau D, Wieser E, Taylor J, Berg S, Smith NJ, Kern R, Picus M, Hoyer S, van Kerkwijk MH, Brett M, Haldane A, del Río JF, Wiebe M, Peterson P, Gérard-Marchant P, et al. Array programming with NumPy. Nature. 2020 Sep; 585(7825):357–362. http://dx.doi.org/10.1038/s41586-020-2649-2, doi: 10.1038/s41586-020-2649-2.

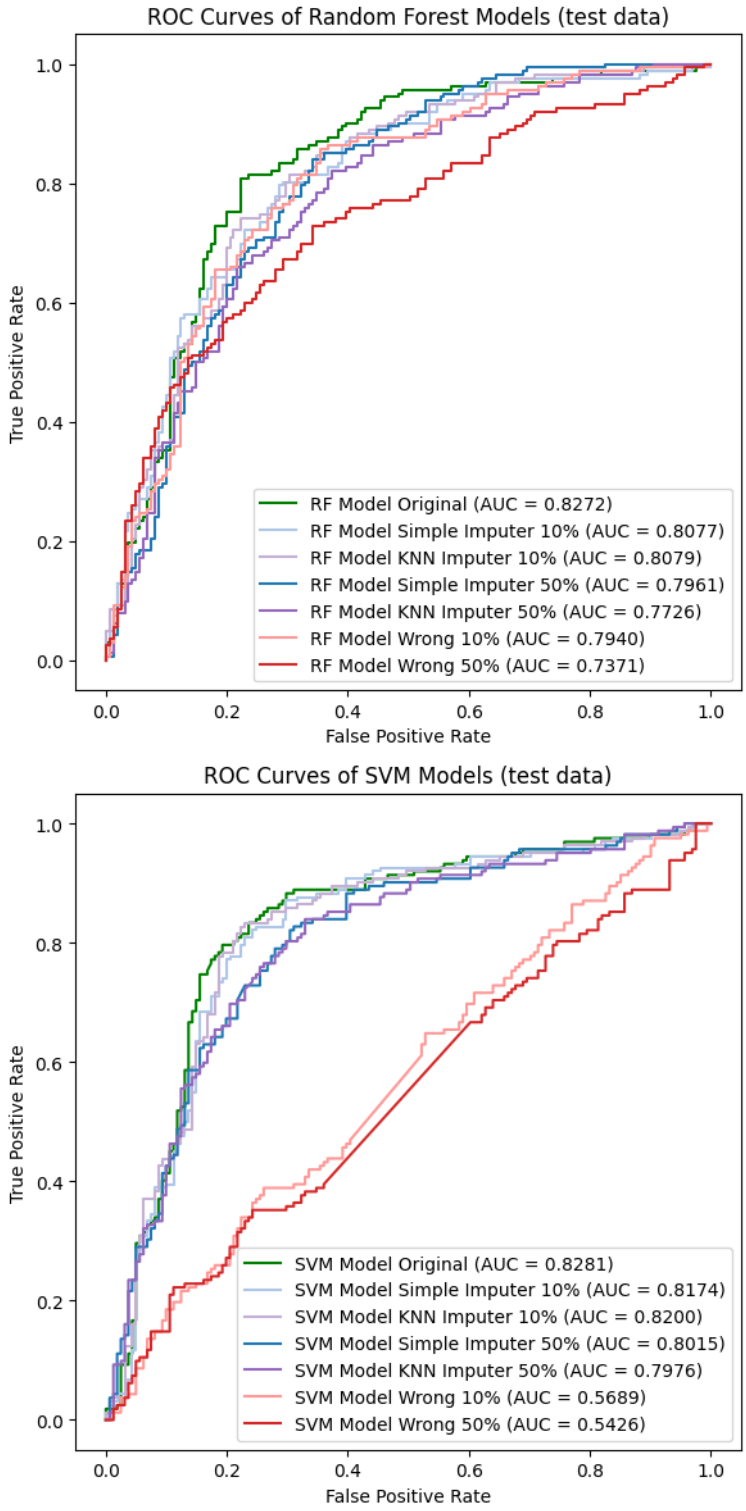
Mahadevan S, Shah SL, Marrie TJ, Slupsky CM. Analysis of Metabolomic Data Using Support Vector Machines. Analytical Chemistry. 2008 Sep; 80(19):7562–7570. http://dx.doi.org/10.1021/ac800954c, doi: 10.1021/ac800954c.

Pedregosa F, Varoquaux G, Gramfort A, Michel V, Thirion B, Grisel O, Blondel M, Müller A, Nothman J, Louppe G, Prettenhofer P, Weiss R, Dubourg V, Vanderplas J, Passos A, Cournapeau D, Brucher M, Perrot M, Édouard Duchesnay, Scikit-learn: Machine Learning in Python; 2018. https://arxiv.org/abs/1201.0490.

Supplementary Figures



Appendix 0—figure A1. Overview of the dataset dirtying and cleaning workflow. All datasets that were used in the model evaluation are colored in yellow. The two datasets crossed out were empty after using the Drop Rows cleaning method.



Appendix 0—figure A2. ROC curves for our two models—Random Forest (top) and linear-kernel SVM (bottom). Each colored line represents one of the seven data conditions tested. The legend in each plot shows its corresponding AUC score.

Supplementary Tables

Appendix 0—table A1. Performance of Random Forest Models with Different Data Conditions

Model Variant	AUC (Train)	AUC (Test)	Accuracy
RF Model Original	0.9885	0.8272	0.7678
RF Model Simple Imputer 10%	0.9909	0.8077	0.7430
RF Model KNN Imputer 10%	0.9892	0.8079	0.7554
RF Model Simple Imputer 50%	0.9780	0.7961	0.7245
RF Model KNN Imputer 50%	0.9935	0.7726	0.7090
RF Model Wrong 10%	0.9957	0.7940	0.7337
RF Model Wrong 50%	1.0000	0.7371	0.6811

Appendix 0—table A2. Top 5 feature importances for Random Forest under the 7 different data set conditions.

RF – Simple Imputer (10%)

Rank	Metabolite	Importance
1	M141	0.0519
2	M142	0.0489
3	M10	0.0354
4	M2	0.0291
5	M15	0.0289

RF – KNN Imputer (10%)

Rank	Metabolite	Importance
1	M142	0.0646
2	M141	0.0439
3	M10	0.0374
4	M2	0.0290
5	M15	0.0273

RF – Simple Imputer (50%)

Rank	Metabolite	Importance
1	M142	0.0495
2	M141	0.0391
3	M7	0.0341
4	M10	0.0330
5	M35	0.0313

RF – KNN Imputer (50%)

Rank	Metabolite	Importance
1	M142	0.0449
2	M141	0.0432
3	M35	0.0270
4	M139	0.0228
5	M10	0.0215

RF – Wrong (10%)

Rank	Metabolite	Importance
1	M141	0.0616
2	M142	0.0587
3	M2	0.0281
4	M152	0.0259
5	M7	0.0250

RF – Wrong (50%)

Rank	Metabolite	Importance
1	M141	0.0262
2	M7	0.0202
3	M142	0.0170
4	M10	0.0165
5	M161	0.0145

RF – Original

Rank	Metabolite	Importance
1	M141	0.0588
2	M142	0.0577
3	M10	0.0362
4	M2	0.0335
5	M7	0.0325

Appendix 0—table A3. Performance of SVM Models with Different Data Conditions

Model Variant	AUC (Train)	AUC (Test)	Accuracy
SVM Model Original%	0.8769	0.8281	0.8281
SVM Model Simple Imputer 10%	0.8773	0.8174	0.8174
SVM Model KNN Imputer 10%	0.8745	0.8199	0.8199
SVM Model Simple Imputer 50%	0.8835	0.8015	0.8015
SVM Model KNN Imputer 50%	0.8677	0.7976	0.7976
SVM Model Wrong 10%	0.8161	0.5689	0.5688
SVM Model Wrong 50%	0.8029	0.5426	0.5426

Appendix 0—table A4. Top 5 feature importances for SVM Model under the 7 different data set conditions.

SVM – Simple Imputer (10%)

Rank	Metabolite	Importance
1	M10	0.1089
2	M2	0.0855
3	M141	0.0835
4	M142	0.0774
5	M152	0.0753

SVM – KNN Imputer (10%)

Rank	Metabolite	Importance
1	M10	0.1051
2	M141	0.0834
3	M142	0.0826
4	M2	0.0821
5	M152	0.0772

SVM – Simple Imputer (50%)

Rank	Metabolite	Importance
1	M10	0.1145
2	M142	0.0928
3	M141	0.0891
4	M7	0.0786
5	M2	0.0713

SVM – KNN Imputer (50%)

Rank	Metabolite	Importance
1	M141	0.1016
2	M142	0.0948
3	M10	0.0918
4	M7	0.0696
5	M15	0.0638

SVM – Wrong (10%)

Rank	Metabolite	Importance
1	M105	0.0676
2	M24	0.0675
3	M136	0.0669
4	M1	0.0658
5	M63	0.0623

SVM – Wrong (50%)

Rank	Metabolite	Importance
1	M101	0.2349
2	M126	0.2030
3	M80	0.1847
4	M105	0.1783
5	M56	0.1778

SVM – Original

Rank	Metabolite	Importance
1	M10	0.1061
2	M141	0.0921
3	M142	0.0913
4	M2	0.0896
5	M18	0.0762